

LOG430 – Labo 01

Nom : Amjad Lekhdar **Cours :** LOG430 – Architecture logicielle **Laboratoire :** Labo 01 – Client/Serveur, Persistence (DAO/RDBMS/NoSQL) **Date :**

Question 1

Énoncé

Quelles commandes avez-vous utilisées pour effectuer les opérations **UPDATE** et **DELETE** dans MySQL ? Avez-vous uniquement utilisé Python ou également du SQL ? Veuillez inclure le code pour illustrer votre réponse.

Réponse

Code utilisé

```
def update(self, user):
    """ Update given user in MySQL """
    self.cursor.execute(
        "UPDATE users SET name = %s, email = %s WHERE id = %s",
        (user.name, user.email, user.id)
    )
    self.conn.commit()
    pass

def delete(self, user_id):
    """ Delete user from MySQL with given user ID """
    self.cursor.execute("DELETE FROM users WHERE id = %s", (user_id,))
    self.conn.commit()
    pass

def delete_all(self): # extra
    """ Empty users table in MySQL """
    self.cursor.execute("DELETE FROM users")
    self.conn.commit()
    pass
```

Explication

J'ai utilisé des requêtes SQL exécutées à travers du code Python à l'aide de la bibliothèque **mysql-connector-python**.

Question 2

Énoncé

Quelles commandes avez-vous utilisées pour effectuer les opérations dans MongoDB ? Avez-vous uniquement utilisé Python ou également du SQL ? Veuillez inclure le code pour illustrer votre réponse.

Réponse

Code utilisé

```
def select_all(self):
    """ Select all users from MongoDB """
    documents = self.collection.find()
    return [User(document["_id"], document["name"], document["email"])
            for document in documents]

def insert(self, user):
    """ Insert given user into MongoDB """
    result = self.collection.insert_one({"name": user.name, "email":
user.email})
    return result.inserted_id

def update(self, user):
    """ Update given user in MongoDB """
    self.collection.update_one(
        {"_id": user.id},
        {"$set": {"name": user.name, "email": user.email}},
    )

def delete(self, user_id):
    """ Delete user from MongoDB with given user ID """
    self.collection.delete_one({"_id": user_id})
```

Explication

J'ai utilisé des commandes fournies par la bibliothèque `pymongo` afin d'effectuer les opérations MongoDB.

Question 3

Énoncé

Comment avez-vous implémenté votre `product_view.py` ? Est-ce qu'il importe directement la `ProductDAO` ? Veuillez inclure le code pour illustrer votre réponse.

Réponse

Code utilisé

```

from models.product import Product
from controllers.product_controller import ProductController

class ProductView:
    @staticmethod
    def show_options():
        """ Show menu with operation options which can be selected by the
user """
        controller = ProductController()
        while True:
            print("\n1. Montrer la liste de produits\n2. Ajouter un
produit\n3. Quitter l'appli")
            choice = input("Choisissez une option: ")

            if choice == '1':
                products = controller.list_products()
                ProductView.show_products(products)
            elif choice == '2':
                name, brand, price = ProductView.get_inputs()
                product = Product(None, name, brand, price)
                controller.create_product(product)
            elif choice == '3':
                controller.shutdown()
                break
            else:
                print("Cette option n'existe pas.")

    @staticmethod
    def show_products(products):
        """ List products """
        print("\n".join(f"{product.id}: {product.name} ({product.brand}) -
${product.price}" for product in products))

    @staticmethod
    def get_inputs():
        """ Prompt user for inputs necessary to add a new product """
        name = input("Nom du produit : ").strip()
        brand = input("Marque du produit : ").strip()
        price = float(input("Prix du produit : ").strip())
        return name, brand, price

```

Explication

J'ai implémenté mon `product_view.py` de manière similaire à `user_view.py`. Il y a méthode `show_option` qui contient le code qui affiche le menu possédant les options, soit l'affichage de la liste des produits, l'ajout des produits et l'arrêt de l'application.

le `product_view.py` n'importe pas directement `ProductDAO`. La vue communique avec `ProductController`, qui lui interagit avec `ProductDAO`. Cette approche provient de l'architecture MVC et

celle-ci permet une meilleure séparation des responsabilités.

Question 4

Énoncé

Si nous devons créer une application permettant d'associer des achats d'articles aux utilisateurs (**Users** → **Products**), comment structurerions-nous les données dans MySQL par rapport à MongoDB ?

Réponse

Pour structurer une application qui associe des achats d'articles aux utilisateurs, je divise les responsabilités entre les utilisateurs, les produits, l'inventaire, le panier et les achats.

Dans MySQL, je propose une structure relationnelle avec plusieurs tables. La table **products** existe afin de garder les informations générales comme le nom, la description et le prix de base. La table **inventory** a pour rôle de gérer les quantités réellement disponibles pour chaque produit. Ensuite, un utilisateur pourrait ajouter des produits dans un panier aussi appelé **cart**, avec une table **cart_items** pour représenter les articles présents dans le panier. Lorsqu'un achat est confirmé, les informations seraient transférées vers les tables **purchases** et **purchase_items**, ce qui permet de garder un historique des achats.

Dans MongoDB, l'approche peut être plus orientée documents. Les produits sont gardés dans une collection **products**, avec un champ ou un sous-document pour l'inventaire. Le panier peut être intégré dans le document de l'utilisateur ou enregistré dans une collection séparée **carts**. Les achats confirmés pourraient être conservés dans une collection **purchases** afin de garder un historique.

Structure possible avec MySQL

```
users
- id
- name
- email

products
- id
- name
- description
- base_price

inventory
- id
- product_id
- quantity_available

carts
- id
- user_id
```

```
- status

cart_items
- id
- cart_id
- product_id
- quantity

purchases
- id
- user_id
- purchase_date
- total_price

purchase_items
- id
- purchase_id
- product_id
- quantity
- unit_price
```

Structure possible avec MongoDB

```
{
  "_id": "user_001",
  "name": "Amjad Lekhdar",
  "email": "amjad@email.com",

  "cart": {
    "items": [
      {
        "product_id": "prod_101",
        "name": "Laptop",
        "quantity": 1,
        "unit_price": 1200
      },
      {
        "product_id": "prod_205",
        "name": "Mouse",
        "quantity": 2,
        "unit_price": 40
      }
    ]
  },

  "purchases": [
    {
      "purchase_id": "purchase_001",
      "purchase_date": "2026-05-20",
      "total_price": 1280,
    }
  ]
}
```

```
"items": [  
  {  
    "product_id": "prod_101",  
    "name": "Laptop",  
    "quantity": 1,  
    "unit_price": 1200  
  },  
  {  
    "product_id": "prod_205",  
    "name": "Mouse",  
    "quantity": 2,  
    "unit_price": 40  
  }  
]  
}
```

Comparaison

La différence principale est que MySQL favorise les relations entre plusieurs tables avec des clés étrangères, tandis que MongoDB permet davantage d’imbriquer les données dans des documents. MySQL serait plus structuré pour gérer les relations et l’intégrité des données, alors que MongoDB serait plus flexible pour représenter des données liées à un utilisateur.

Conclusion
